Portfolio Studio and Photo Processor

Requested Tasks, Implementation, QA, and Repository Delivery Report

28 October 2026

1. Requested Work and Delivery Status

The requested implementation is complete in source, with verified Linux application workflows and published Git history. macOS and Windows installation acceptance, signing, and target-hardware checks remain open. The existing Python application remains available.

The user requested execution of Portfolio_Studio_Electron_Plan.md through eight separate subagents, working one at a time. The supervising agent was to review each output before starting the following step. The two new apps were to occupy separate new folders under design/tools. Later requests authorized committing, merging to master, and pushing both the working branch and master.

Delivered Locations

• Portfolio Studio: tools/portfolio-studio-electron. Photo Processor: tools/photo-processor-electron.

• Shared React, TypeScript, Electron services, processing, and QA: tools/portfolio-electron-shared.

• Workflow and verification documentation: docs/electron. Native build automation: .github/workflows/electron-apps.yml.

Execution Control

Eight subagents performed the eight numbered steps sequentially. The supervisor inspected source, tested outputs, reviewed screenshots, and recorded approval between steps. A capacity error in Step 4 and a workspace-credit interruption in Step 8 were resumed using the same assigned agents. No replacement agent or parallel step execution was used.

Implementation and QA evidence is dated 28 October 2026. This report uses America/Los_Angeles for user-facing dates. No exact wall-clock labor duration is asserted.

2. Results of the Eight Numbered Steps

Step 1: Define the Apps

Inspected the actual Photography, Engineering, and Archive catalogs; defined ownership and typed models; preserved extension fields, framing, cache versions, ordering, rich text, mixed media, and Archive templates. Defined publishedImage: true to identify the thumbnail as the sole public image while retaining file as the stable photo key.

Step 2: Shared Architecture

Created independent React/TypeScript/Vite entry points and Electron Forge packaging, npm workspaces, lockfile, launchers, sandboxed renderers, context isolation, narrow validated IPC, scoped file access, and a worker-thread processing boundary. Package inspection found omitted native dependencies; corrected packaging and verified package-local Sharp/libvips resolution.

Step 3: Optional Linking

Added explicit companion selection and enablement, authenticated versioned loopback messages, private credentials and workspace/task validation, per-app single-instance locks, private source records, and publication receipts. Both apps remain usable independently; failed optional communication does not prevent startup.

Step 4: Interfaces and Editors

Built consistent navigation, contextual panels, status, keyboard resizing, shortcuts, undo/redo, and Light/Dark/System themes. Added Studio management editors and Processor queue/recipe controls. Preserved dirty edits during refreshes and switching, and sanitized rich-text paste.

Steps 5 and 6: Processing and Protection

Connected the original-to-encoded-image pipeline, exact-byte previews, private render tokens, source-hash checks, progress, cancellation, queue rendering, drafts, relinking, and bundled HEIC decoding. Applied adaptive tiled/corner watermarks and verified fresh EXIF/XMP rights notices in WebP, JPEG, and PNG. Tiled base strength was reduced from 7/255 to 6/255.

Steps 7 and 8: Publication and Verification

Implemented real Save Locally and Add to Photo Catalog actions, journaled public/private transactions, backups, writer coordination, external-change checks, reconciliation, recovery, restricted website previews, and deliberate legacy migration/cleanup. Delivered reusable browser, packaged-worker, native-window, crash-recovery, and benchmark scripts, plus native CI and release documentation.

3. Functional and Safety Outcomes

Portfolio Studio

Studio manages descriptions, tags, visibility, prominence, manual order, position locks, and reviewable arrangement suggestions. Engineering supports rich text and mixed image/video media. Archive supports collections and per-image presentation. Save Locally updates catalogs and their generated website HTML, settings, or photography JavaScript together.

Photo Processor

Processor imports read-only originals and applies orientation correction, rotation, straightening, cropping, resizing, watermarking, metadata, and encoding outside the interface thread. Defaults are an 800-pixel maximum side and WebP quality 78. Other supported outputs are JPEG and PNG. Render Queue Previews processes sequentially; publication is explicit for the selected photograph.

The preview displays actual encoded bytes. A private token identifies those same bytes for publication. Recipe or source changes require another render. Published output stays visible for inspection, while its consumed token prevents duplicate publication. Reprocessing starts from the original and saved recipe.

Privacy, Protection, and Recovery

Original paths, recipes, integration tokens, journals, and backups are stored outside the selected website repository. Raw rasterization strips source metadata; exports receive allowlisted authorship and usage notices. Parsed metadata checks covered Artist, XPAuthor, Copyright, UserComment, and XMP/PLUS restrictions. These notices communicate restrictions and do not technically prevent AI training. No adversarial pixel-protection feature was added.

Transactions preserve before/after bytes privately and reject unexpected file changes. Recovery verifies dead ownership, takes a separate exclusive recovery claim, checks backup and destination hashes, and refuses unrelated external edits. Review identified and corrected a concurrent-recovery race. A process killed during recovery itself can leave a conservative claim requiring manual private-state review.

Legacy paired entries continue displaying. Migration retains full-size files; cleanup is a separate confirmed action blocked by remaining references or saved/active original use. Previous generated single-image exports are replaced safely when formats change. QA did not migrate or delete production photographs.

4. Verification and Performance Evidence

Passing Checks

• TypeScript checks; both production Vite builds; 27 Electron unit/integration cases; real killed-writer recovery; concurrent writer and recovery exclusion.

• Existing website tests, including 330 photography layout cases and homepage preview behavior; live portfolio validation; six validation unit cases; six targeted legacy catalog-sync cases.

• Both independent Linux Forge packages; package-local Sharp resolution; bundled HEIC decode of a 32 × 24 HEVC fixture; encoded EXIF/XMP notices.

• Real packaged Linux windows under a temporary Xvfb display: independent launch, sandboxed preload, optional linked delivery, encoded rendering, publication receipt, and Studio refresh.

• Service-backed Chromium workflows saved Photography, Engineering, and Archive; published a new image in a disposable workspace; retained output inspection; blocked token reuse; and rendered valid legacy and new website images.

Observed Linux Measurements

• Packaged launch to bridge-ready window: Studio 879 ms; Processor 656 ms.

• Built renderer readiness: 240 ms. Crop-toggle interaction, including automation round trip: 53 ms.

• Sixty foreground scrolling frames: 988 ms, approximately 60 fps for the tested small catalog.

• Ten sequential synthetic 12 MP JPEG renders: 3.83 seconds total, 356–420 ms per photograph.

These are single observed Linux runs on four available CPUs with Node 24.19.0, Electron 38.8.6, Sharp 0.34.5, and libvips 8.17.3. The batch uses a flat-color 4000 × 3000 JPEG, 90% crop, 800-pixel WebP quality 78, watermarks, and metadata. It does not measure publication time or complex real-photo compression. Crop timing is a control response, not live drag rendering. No cross-platform performance acceptance is claimed.

Nonpassing or Limited Checks

The broad legacy Python suite ran 134 tests and did not pass: one assertion failed, 62 errored, and two skipped. Most errors required a Tk display or writable home-directory backups; observed baseline issues involved Archive newline snapshots and a preference assertion omitting appearance. Targeted affected checks passed. Linux packaged-worker runs emitted GLib teardown warnings while returning successful results. The initial display-free native launch failed; the Xvfb run passed.

5. Commits, Merge Resolution, and Push Delivery

Initial Commit and Local Master

Committed the implementation as 04388eb, “Add independent Electron portfolio tools and safe publication workflows.” That commit contained 97 files, 34,247 insertions, and three deletions. Created local master from the then-known origin/master and fast-forwarded it to include the implementation. The local working tree was clean.

Remote Changes and Conflict Resolution

The first push of work and master was rejected because both remote branches had newer history. Fetched the remote branches and merged origin/master without force-pushing. Resolved conflicts in .gitignore, the deleted home-previews.js, and catalog_tools.py.

Preserved the remote website-script consolidation. Removed the obsolete home-previews.js and carried its single-image guard into home.js. Retained the single-image behavior in consolidated photography.js and the canonical floating-ratio generation in catalog_tools.py. Combined Electron ignore entries with the remote cleanup rules.

After reconciliation, reran TypeScript, all 27 Electron cases, consolidated website tests, and live catalog validation successfully. The final Linux package, browser, and performance evidence in Section 4 predates this merge; those full packaged and visual suites were not repeated after reconciliation.

Final Published Repository State

Created merge commit c99dfa0, “Merge latest master while preserving Electron publication compatibility.” Confirmed origin/work was already included, fast-forwarded local work to master, and pushed both branches successfully. Remote verification returned the same full hash for master and work: c99dfa06be5a8956cea917197d3d4f32eda2f117. No force-push was used.

The repository was clean at report preparation. This new report is stored outside the repository in /workspace/reports and is not included in those commits or pushes. GitHub Actions configuration was published, but remote run outcomes were not inspected and are not asserted here.

6. Remaining Release Actions and Supporting Records

Open Native Release Gates

• Run and review macOS and Windows CI jobs, package artifacts, screenshots, and codec checks. Configuration alone does not establish successful execution.

• Install or extract each delivered package on supported target architectures; test independent and linked launch, real HEIC import/reframing, native dialogs, shortcuts, themes, resizing, conflicts, and recovery.

• Exercise Windows Squirrel installation, shortcuts, update, and uninstall, and macOS application launch. The native smoke script launches packaged executables; it does not run installers.

• Apply owner-provided signing/notarization credentials and recheck signed outputs. Current build configuration produces unsigned development artifacts unless credentials are supplied.

• Measure representative real-photo workloads on target hardware. Retain the Python application and existing content until native acceptance is complete.

Launch and Reproduction

From the design repository root, install Node.js 22 or later and run npm ci. Launch Studio with npm run start:studio and Processor with npm run start:processor. Each app also has Launch.command and Launch.bat. Installed packages bundle their runtime and do not require Python or a separately configured server.

Reusable verification commands include npm run typecheck, npm run test:electron, npm run build:electron, npm run qa:browser, npm run package:electron, npm run qa:packages, npm run qa:native, and npm run benchmark:electron. Browser checks require Chromium; native Linux checks require a display.

Supporting Records Reviewed

• docs/electron/IMPLEMENTATION_PLAN.md: preserved user-supplied eight-step plan.

• docs/electron/EXECUTION_QA.md: sequential review gates, supervisor evidence, and legacy-suite limitations.

• docs/electron/README.md and RELEASE_VERIFICATION.md: operation, reproduction, measured results, and native acceptance checklist.

• docs/electron/qa-results/linux-native.json, linux-browser.json, and linux-batch.json: machine-readable observations.

• tools/portfolio-electron-shared/ARCHITECTURE.md, RESPONSIBILITIES.md, and INTEGRATION.md: architecture, ownership, and integration handoffs.

• Git commits 04388eb and c99dfa0; branch and remote verification recorded during the session. Report basis also includes the session’s tool results and reviewed screenshots.

The packaged Word report template could not be accessed through the available skill resource reader. This report therefore uses a clean custom layout without an invented corporate logo. Its content is an implementation record, not certification of production or native release readiness.